Checking if a union alternative is active

Document #:	P2641R1
Date:	2022-10-08
Project:	Programming Language C++
Audience:	EWG, LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Daveed Vandevoorde <daveed@edg.com>

Contents

§ 1 Introduction

Let's say you want to implement an `Optional<bool>` such that `sizeof(Optional<bool>) == 1`. This is conceptually doable since `bool` only has 2 states, but takes up a whole byte anyway, which leaves 254 bit patterns to use to express other criteria (i.e. that the `Optional` is disengaged).

There are two ways to do this, neither of which have undefined behavior:

Union	Reinterpret

`struct OptBool {`
`  union { bool b; char c; };`
 	
`  OptBool() : c(2) { }`
`  OptBool(bool b) : b(b) { }`
 	
`  auto has_value() const -> bool {`
`  return c != 2;`
`  }`
 	
`  auto operator*() -> bool& {`
	`struct OptBool {`
	`  char c;`
	
	`  OptBool() : c(2) { }`
	`  OptBool(bool b) { new (&c) bool(b); }`
	
	`  auto has_value() const -> bool {`
	`  return c != 2;`
	`  }`
	
	`  auto operator*() -> bool& {`

Union	Reinterpret

`    return b;`
`  }`
`};`
	`    return (bool&)c;`
	`  }`
	`};`

Both of these are fine: `operator*` works because we are returning a reference to a very-much-live `bool` in both cases, and `has_value()` works because we are allowed read through a `char` (explicitly, per [basic.lval]/11.3[1]).

However, try to make this work in `constexpr` and we immediately run into problems.

The union solution has a problem with `has_value()` because we're not allowed to read the non-active member of a union ([expr.const]/5.9). `c` isn't the active member of that union, so we can't read from it, even though it's `char`.

The reinterpet solution has two problems:

— the placement new into `&c` doesn't work. Placement-new is explicitly disallowed by [expr.const]/5.17. While we have `std::construct_at()` now to work around the lack of placement-new, that API is typed and would require a `bool*`, which we don't have.

— `operator*()` involves a `reinterpret_cast`, which is explicitly disallowed by [expr.const]/5.15.

This is unfortunate - we have a well-defined runtime solution that we cannot use during constant evaluation time. It'd be nice to do better.

§ 2 Proposal

Supporting the reinterpret solution is expensive - since it requires the implementation to track multiple types for the same bytes. But supporting the union solution, because it's more structured (and thus arguably better anyway), is very feasible. After all, our problem is localized to `has_value()`.

During runtime, we do need to read from `c`. But during constant evaluation time, we actually don't. What we need `c` for is to know whether `b` is the active member of the union or not. But the compiler already knows whether `b` is the active member or not. Indeed, that is precisely *why* it's rejecting this implementation. What if we could simply ask the compiler this question?

```
struct OptBool {
  union { bool b; char c; };

  constexpr OptBool() : c(2) { }
  constexpr OptBool(bool b) : b(b) { }

  constexpr auto has_value() const -> bool {
    if consteval {
      return std::is_active_member(&b);
    } else {
      return c != 2;
    }
```

```
  }

  constexpr auto operator*() -> bool& {
    return b;
  }
};
```

There's no particular implementation burden here - the compiler already needs to know this information. No language change is necessary to support this change either.

2.1 Interesting Cases

Let's go over some interesting cases to flesh out how this facility should behave

2.1.1 Nested Anonymous Unions

Consider:

```
union Outer {
  union {
    int x;
    unsigned int y;
  };
  float f;
};
consteval bool f(Outer *p) {
  return std::is_consteval_active_member(&p->x);
}
```

If the active member of `p` is actually `f`, then not only is `p->x` not the active member of *its* union but the union that it's a member of isn't even itself an active member. What should happen in this case?

Arguably, this should be valid and return `false`. It doesn't matter how many layers of inactivity there are - `p->x` isn't the active member and that's what we're asking about.

2.1.2 Subobject

Consider:

```
struct S {
  union {
    struct {
      int i;
    } n;
  };
};

consteval bool f(S s) {
  return std::is_consteval_active_member(&s.n.i);
}
```

Here, `s.n.i` is not a variant member of a union - it's a subobject of `s.n`. We could say this is ill-formed, requiring that the pointer into this function actually be a pointer to a variant member. But that's seems overly strict. After all, the specific language rule is, emphasis mine:

(5.9) an lvalue-to-rvalue conversion that is applied to a glvalue that refers to a non-active member of a union **or a subobject thereof;**

The same rule that would reject using `s.n` if that's not an active member would reject `s.n.i` if it's not the subobject of an active member. So we should just accept this case (returning true of `&s.n` is the active member), rather than rejecting it.

2.1.3 Other rules

There's a similar rule in this space, [expr.const]/5.8

(5.8) an lvalue-to-rvalue conversion unless it is applied to

(5.8.1) — a non-volatile glvalue that refers to an object that is usable in constant expressions, or

(5.8.2) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E;

Should the facility in this paper address this case as well? At this point, this would become a "is this a pointer that I can read during constant evaluation?" facility. Would that actually be useful? It's not clear what you could do with this information.

2.2 Alternative Naming

Should the name reflect that this facility can only be used during constant evaluation (`std::is_consteval_active_member`) or not (`std::is_active_member`)?

It would help to make the name clearer from the call site that it has limited usage. But it is already a `consteval` function, so it's not like you could misuse it.

2.3 Implementation Experience

Daveed Vandevoorde has implemented this proposal in EDG.

As pointed out by Johel Ernesto Guerrero Peña and Ed Catmur, this facility basically already works in gcc and clang, which has a builtin function (`__builtin_constant_p`) that be used to achieve the same behavior.

3 Wording

Add to 21.3.3 [meta.type.synop]:

```
// all freestanding
namespace std {
```

```
  // ...

  // [meta.const.eval], constant evaluation context
  constexpr bool is_constant_evaluated() noexcept;
+ template<class T>
+   consteval bool is_active_member(T*);
}
```

And add to 21.3.11 [meta.const.eval] (possibly this section should just change name, but it feels like these two should just go together):

```
constexpr bool is_constant_evaluated() noexcept;
```

1 *Effects*: Equivalent to:

```
if consteval {
  return true;
} else {
  return false;
}
```

2 [*Example 1*:

```
constexpr void f(unsigned char *p, int n) {
  if (std::is_constant_evaluated()) {           // should not be a constexpr
        if statement
    for (int k = 0; k<n; ++k) p[k] = 0;
  } else {
    memset(p, 0, n);                            // not a core constant
        expression
  }
}
```

— *end example*]

```
template<class T>
  consteval bool is_active_member(T* p);
```

3 *Returns*: `true` if `p` is a pointer to the active member of a union or a subobject thereof; otherwise, `false`.

4 *Remarks*: A call to this function is not a core constant expression ([expr.const]) unless `p` is a pointer to a variant member of a union or a subobject thereof.

5 [*Example 2*:

```
struct OptBool {
  union { bool b; char c; };

  constexpr OptBool() : c(2) { }
  constexpr OptBool(bool b) : b(b) { }

  constexpr auto has_value() const -> bool {
    if consteval {
```

```
      return std::is_active_member(&b);   // during constant evaluation,
        cannot read from c
    } else {
      return c != 2;                      // during runtime, must read from
        c
    }
  }

  constexpr auto operator*() -> bool& {
    return b;
  }
};

constexpr OptBool disengaged;
constexpr OptBool engaged(true);
static_assert(!disengaged.has_value());
static_assert(engaged.has_value());
static_assert(*engaged);
```

— *end example*]

§ 4 References

[N4868] Richard Smith. 2020-10-18. Working Draft, Standard for Programming Language C++. https://wg21.link/n4868

---

1. All references to this paper are to [N4868], C++20 for convenient and stable linking.↩